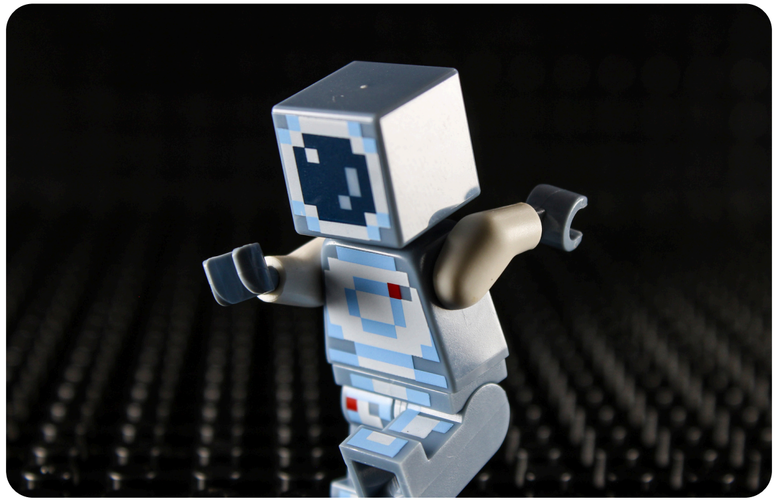
# DEVSETUP → AI



FROM INSTALLATION TO IMPLEMENTATION – ONE AI AGENT, ZERO SETUP STRUGGLES.

# THE HIDDEN BARRIER TO SOFTWARE DEVELOPMENT: ENVIRONMENT SETUP

## The problem:

- Complex installations: Students must independently download and install multiple IDEs, compilers, SDKs, drivers and libraries.
- Configuration difficulties: Setting up board packages, dependencies, environment variables and hardware connections can be confusing.
- Debugging dependency issues: Beginners struggle to distinguish coding errors from environment and configuration errors.
- Fragmented workflow: Users switch between tutorials, documentation, terminal commands and AI assistants to complete a single project.
- Lost productivity: Time that should be spent learning, coding and innovating is wasted on repetitive technical setup.

***Our target users***

Engineering students | IoT developers | Beginners | Researchers | Software developers

The core problem:
**"AI can write your code, but who prepares your computer to run it?"**

# DEVSETUP AI – YOUR AUTONOMOUS DEVELOPMENT ENVIRONMENT AGENT



**What are we building?**

An AI-powered agent that automates the development journey, from understanding project requirements to configuring the environment, generating code, executing it and troubleshooting errors.

## HOW IT WORKS?

**Understand**-User describes the project in natural language.

**Detect**-Analyses the operating system, installed software, dependencies and available hardware.

**Example use case:**
"I want to build a Bluetooth-controlled LED project using ESP32."
DevSetup AI checks the environment, configures Arduino and ESP32 dependencies, generates the program, compiles it and guides the user through testing.

**Develop**-Generates project code, required files and configurations

**Execute & Debug**-Compiles or runs the program, diagnoses errors and attempts fixes.

**Setup**-Identifies missing tools and installs/configures them with user approval.

**Key differentiators:**

- Environment-aware AI automation.
- Setup-to-execution workflow in one agent.
- Context-aware error diagnosis.
- Personalised assistance for beginners.
- Integration with Nonilion's virtual office.

# EXISTING → GAP

## CURRENT SETUP

- Code generation: write functions, files, and whole projects from a prompt
- Editing and refactoring: modify existing code across multiple files
- Debugging help: explain errors and suggest fixes
- Some terminal access: tools like Claude Code, Cursor, and Copilot agents can run commands and install packages.

## INTENDED SCOPE OF RESOLUTION

- A unified, guided workflow that connects the whole chain
- Integration, not invention: we combine capabilities that exist separately into one specialised workflow
- Context sharing: detection results inform setup, and setup results inform troubleshooting
- Guided experience: the user is walked through the process and approves risky steps
- Reproducibility: the final output can include a setup script or lockfile so the setup can be repeated

**DISTINCTION FROM TRADITIONAL AI:**

**NOT JUST REACTIVE, BUT PROACTIVE.**

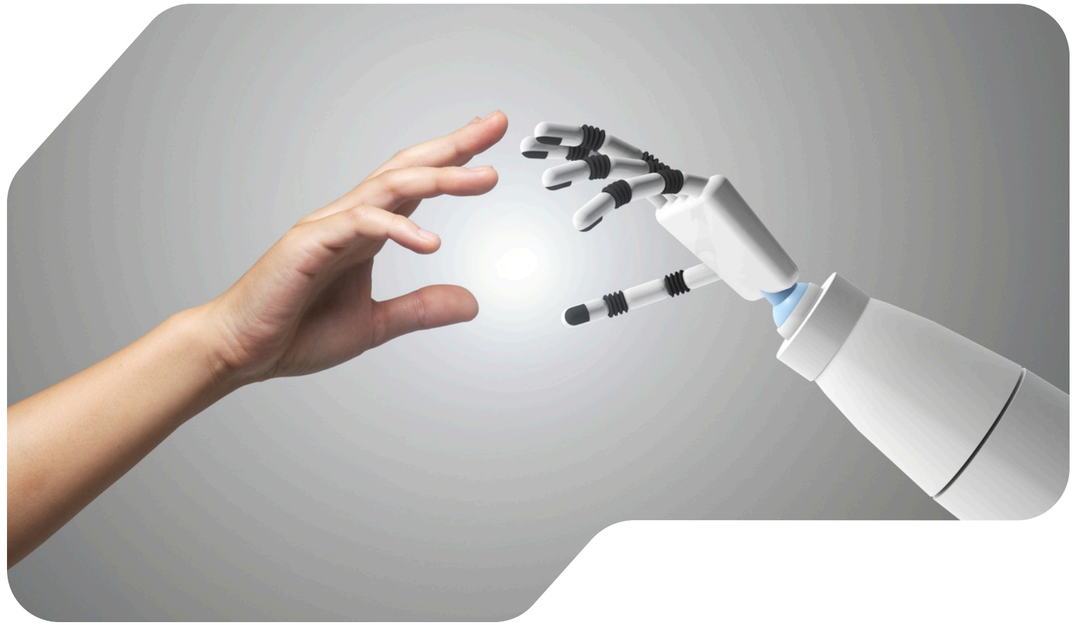


CLOUDLET

# THE REAL WORLD

**AUTONOMOUS VEHICLES NAVIGATING WITHOUT HUMAN INTERVENTION.**

ROBOTIC WAREHOUSE SYSTEMS MANAGING INVENTORY AND DELIVERIES.



PERSONAL DIGITAL AGENTS THAT MANAGE SCHEDULES, RESOURCES, AND DECISIONS.

AI RESEARCH ASSISTANTS PROPOSING EXPERIMENTS OR SOLUTIONS.

# FEASIBILITY → WORKABILITY

## TECHNICAL FEASIBILITY

- Scan the system: Python can read the OS, CPU, RAM, and disk space
- Detect GPU and drivers: existing tools like nvidia-smi can be run and read
- Run commands: Python can install software and capture any errors
- Use package managers: pip, npm, conda, and others already work from the command line
- AI planning: LLM APIs can read errors and suggest the next step.

## OPERATIONAL FEASIBILITY

- Chat interface: the user describes what they want to set up through Nonilion
- Local desktop companion: runs on the user's computer to see the real hardware and software
- Simple flow: user asks → system is scanned → plan is proposed → user approves → setup runs → errors are fixed
- Safety: user approval for risky steps, isolated environments, and a log of every action

## ECONOMICAL FEASIBILITY

- Low starting cost: open-source tools handle scanning, running commands, and package management
- No model training: we use existing AI APIs and pay only for what we use
- Main costs: AI API usage, hosting for the chat backend, and testing on different systems

# Turning Development Automation into a Scalable Business

## Revenue streams

**1. Freemium subscription**

Free basic environment detection and limited setup assistance; paid plans for advanced automation and higher usage.

**2. Institutional licensing**

Annual subscriptions for colleges, universities and technical training institutes.

**3. Enterprise solutions**

Customised development environment automation for software companies and engineering teams.

**4. API and integration services**

Paid integrations, custom environment templates and automation workflows for other platforms.

**Target customers:**

- Engineering colleges and universities.
- Individual students and developers.
- Coding bootcamps and training institutes.
- Software development companies.
- Corporate engineering teams.

**Value proposition:**

- Reduces repetitive setup effort.
- Simplifies development for beginners.
- Improves developer productivity.
- Reduces environment-related troubleshooting.
- Creates a standardised development workflow.